

```

1 import numpy as np
2 import pandas as pd
3 import yfinance as yf
4 import matplotlib.pyplot as plt
5
6 from tensorflow.keras.models import Sequential
7 from tensorflow.keras.layers import LSTM, Dense, Dropout, BatchNormalization
8 from tensorflow.keras.callbacks import EarlyStopping
9 from sklearn.preprocessing import MinMaxScaler
10 from sklearn.metrics import mean_absolute_error
11
12
13 # Download NIFTY 50 data
14 df = yf.download(
15     "^NSEI",
16     start="2015-01-01",
17     end="2026-01-01",
18     auto_adjust=False
19 )
20
21 if isinstance(df.columns, pd.MultiIndex):
22     df.columns = df.columns.get_level_values(0)
23
24 df = df[["Open", "High", "Low", "Close"]].dropna()
25
26
27 # Scale data
28 scaler = MinMaxScaler()
29 data = scaler.fit_transform(df)
30
31
32 # Create sequences
33 look_back = 20
34 X, y = [], []
35
36 for i in range(len(data) - look_back):
37     X.append(data[i:i + look_back])
38     y.append(data[i + look_back, 3])
39
40 X = np.array(X)
41 y = np.array(y)
42
43
44 # Train-test split
45 split = int(len(X) * 0.8)
46
47 X_train = X[:split]
48 X_test = X[split:]
49
50 y_train = y[:split]
51 y_test = y[split:]
52
53
54 # LSTM model
55 model = Sequential([
56     LSTM(64, input_shape=(look_back, 4)),
57     Dropout(0.2),
58     BatchNormalization(),
59     Dense(32, activation="relu"),
60     Dense(1)
61 ])
62
63 model.compile(
64     optimizer="adam",
65     loss="mean_squared_error"
66 )
67
68
69 # Train
70 model.fit(
71     X_train,
72     y_train,
73     epochs=50,
74     batch_size=32,
75     shuffle=False,

```

```

76     callbacks=[
77         EarlyStopping(
78             monitor="loss",
79             patience=5,
80             restore_best_weights=True
81         )
82     ],
83     verbose=1
84 )
85
86
87 # Predict
88 pred = model.predict(X_test, verbose=0).flatten()
89
90
91 # Convert predictions back to original price
92 temp = np.zeros((len(pred), 4))
93 temp[:, 3] = pred
94 pred = scaler.inverse_transform(temp)[:, 3]
95
96 temp = np.zeros((len(y_test), 4))
97 temp[:, 3] = y_test
98 actual = scaler.inverse_transform(temp)[:, 3]
99
100
101 # Calculate MAE
102 mae = mean_absolute_error(actual, pred)
103
104 print("Mean Absolute Error (MAE):", mae)
105
106
107 # Test dates
108 dates = df.index[split + look_back:]
109
110 # Split date
111 split_date = dates[0]
112
113
114 # Plot
115 plt.figure(figsize=(15, 8))
116
117 # All data
118 plt.plot(
119     df.index,
120     df["Close"],
121     label="All Data",
122     alpha=0.5
123 )
124
125 # True values
126 plt.plot(
127     dates,
128     actual,
129     label="True Values",
130     linestyle="--"
131 )
132
133 # Predicted values
134 plt.plot(
135     dates,
136     pred,
137     label="Predicted Values",
138     linestyle="--"
139 )
140
141 # Train/Test split line
142 plt.axvline(
143     x=split_date,
144     linestyle="--",
145     linewidth=2,
146     label="Train / Test Split"
147 )
148
149 # Display MAE on graph
150 plt.text(
151     0.02,
152     0.95,
153     f"MAE: {mae:.2f}",

```

```
154     transform=plt.gca().transAxes,  
155     fontsize=12,  
156     verticalalignment="top"  
157 )  
158  
159 plt.xlabel("Date")  
160 plt.ylabel("NIFTY 50 Close Price")  
161 plt.title("NIFTY 50 LSTM Prediction")  
162  
163 plt.legend()  
164 plt.grid(True)  
165  
166 plt.show()  
167  
168
```

[*****100%*****] 1 of 1 completedEpoch 1/50

C:\Users\Student\.conda\envs\nifty50\Lib\site-packages\keras\src\layers\rnn\rnn.py:199: UserWarning: Do not pass an `input\_shape` argument to `Layer` objects. It is not supported. Use `Layer.build` instead.
super().__init__(**kwargs)

```
68/68 ━━━━━━━━━━━ 2s 4ms/step - loss: 0.0267
Epoch 2/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0261
Epoch 3/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0140
Epoch 4/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0081
Epoch 5/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0068
Epoch 6/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0061
Epoch 7/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0056
Epoch 8/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0052
Epoch 9/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0044
Epoch 10/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0041
Epoch 11/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0043
Epoch 12/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0041
```

1 Start coding or [generate](#) with AI.

```
Epoch 13/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0038
Epoch 15/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0039
Epoch 16/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0043
Epoch 17/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0037
Epoch 18/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0036
Epoch 19/50
68/68 ━━━━━━━━━━━ 0s 4ms/step - loss: 0.0029
Epoch 20/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0032
Epoch 21/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0033
Epoch 22/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0033
Epoch 23/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0030
Epoch 24/50
68/68 ━━━━━━━━━━━ 0s 3ms/step - loss: 0.0036
Mean Absolute Error (MAE): 29902.51028216748
```

NIFTY 50 LSTM Prediction

